Детекция в видео и мультиобъектный трекинг

от покадровых боксов к траекториям с постоянными номерами

Александр Савельев

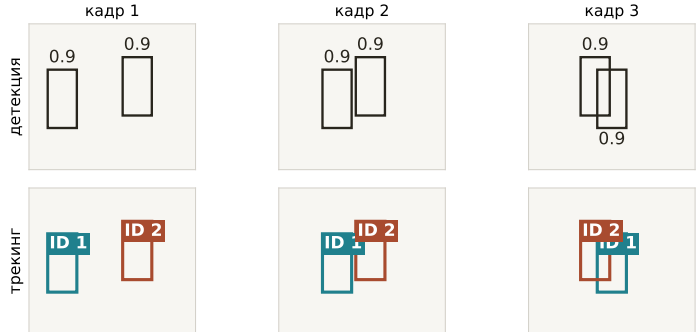
Курс: Обработка видео в контексте машинного обучения и ИИ
Лекция 5

План лекции

Главный вопрос: *как превратить боксы, найденные в каждом кадре отдельно, в траектории, где у каждого объекта один номер на всё видео*.

1. Постановка: чем трекинг отличается от детекции; детекция с учётом соседних кадров
2. Tracking-by-detection: цикл трекера и жизнь трека
3. Модель движения: фильтр Калмана — от сложения двух чисел до матриц
4. Ассоциация: IoU, Махаланобис, внешность; задача о назначениях и венгерский алгоритм
5. Семейство SORT: DeepSORT, ByteTrack, OC-SORT, BoT-SORT, TrackTrack
6. Обучаемая ассоциация: CenterTrack, MOTR, SOT, трекинг точек
7. Метрики: MOTA, IDF1, HOTA — что именно они считают
8. Датасеты, типичные провалы и состояние области в 2026

1. Детекция знает «что», трекинг — ещё и «кто»



в кадре 3 люди поменялись местами: детектор не знает, кто есть кто; трекер сохраняет номера

Интуиция. Детектор в каждом кадре отвечает «здесь человек, уверенность 0.9». Он не знает, что бокс из кадра 3 — тот же человек, что в кадре 1. Трекер добавляет ровно одну вещь: *номер объекта (ID)*, одинаковый во всех кадрах.

1. Формально: детекции кадра и треки

Детекции кадра t

$\mathcal{D}_t = \{d_t^1, \ldots, d_t^{M_t}\}, \quad d_t^j = (b_t^j,\ s_t^j,\ c_t^j)$

$b = (x, y, w, h)$ — бокс: центр (x, y), ширина w, высота h в пикселях; $s \in [0, 1]$ — уверенность детектора (score); c — класс; M_t — сколько боксов в кадре.

Трек и задача MOT (multi-object tracking)

Трек объекта с номером k: $\tau_k = \{(t,\ b_t^k)\ :\ t \in \mathcal{T}_k\}$, где $\mathcal{T}_k$ — кадры, в которых объект виден.

Задача: по видео $X_{1:T}$ найти множество треков $\{\tau_k\}$ — сколько объектов, где каждый в каждом кадре и какой у него номер.

Число объектов заранее неизвестно, объекты появляются и исчезают, закрывают друг друга. Отвечать обычно нужно *онлайн*: решение о кадре t — только по кадрам $1..t$.

1. Мера перекрытия боксов: IoU

IoU (intersection over union) понадобится трижды: связать боксы соседних кадров, построить стоимость в трекере и посчитать метрики.

IoU двух боксов A и B

Углы бокса: $x_1 = x - \frac{w}{2},\ x_2 = x + \frac{w}{2}$ (так же по y).

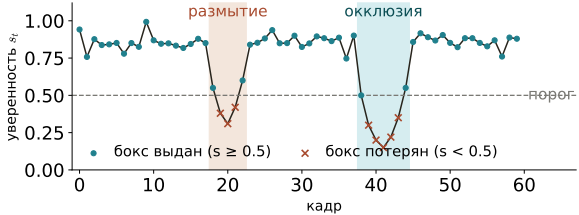
$$w_\cap = \max\bigl(0,\ \min(x_2^A, x_2^B) - \max(x_1^A, x_1^B)\bigr), \qquad h_\cap \text{ — так же по } y$$

$$\mathrm{IoU}(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{w_\cap h_\cap}{w_A h_A + w_B h_B - w_\cap h_\cap} \in [0,1]$$

$\max(0, \cdot)$ — ноль, если боксы не касаются; в знаменателе пересечение вычитаем, так как оно посчитано дважды.

Пример: два бокса 10×20, сдвиг на 5 px по x: $|A \cap B| = 5 \cdot 20 = 100$, $|A \cup B| = 200 + 200 - 100 = 300$, $\mathrm{IoU} = 0.33$. Сдвиг на полширины — уже только треть.

1. Покадровый детектор «мигает»

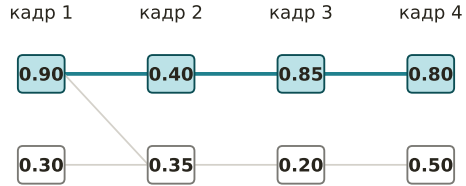


- Уверенность s_t одного и того же объекта скачет: размытие, расфокус, частичная окклюзия, редкая поза.
- Порог на s_t превращает провалы в пропуски: объект «исчезает» на несколько кадров.
- Без связи кадров каждый пропуск — риск, что после него объект получит новый номер.

Два способа использовать время: улучшить сами детекции по соседним кадрам (Seq-NMS, FGFA) или связать детекции в треки (трекер).

1. Seq-NMS: граф боксов

Han et al., 2016



путь 0.90 → 0.40 → 0.85 → 0.80: сумма 2.95 — максимум; бокс кадра 2 (0.40) «подтягивается» до среднего 0.74

Граф боксов

Вершины — боксы всех кадров. Ребро между боксом *i* кадра $t-1$ и боксом *j* кадра *t*, если $\mathrm{IoU}(b^i_{t-1}, b^j_t) \geq 0.5$ — «скорее всего, это один объект». Путь π по графу выбирает по одному боксу в каждом кадре: $\pi(t)$ — номер бокса в кадре *t*.

Объект, уверенно найденный в кадрах 1, 3, 4, «подтягивает» неуверенный бокс кадра 2: у них общий путь.

1. Seq-NMS: лучший путь через динамику

Ищем путь по рёбрам графа с максимальной суммой уверенностей:

$$S^* = \max_{\pi} \sum_{t=1}^{T} s_t^{\pi(t)}, \qquad \mathrm{IoU}\big(b_{t-1}^{\pi(t-1)},\, b_t^{\pi(t)}\big) \geq 0.5 \;\; \forall t$$

Как ищем: рекуррентное соотношение

$S_t(j)$ — лучшая сумма пути, который заканчивается в боксе j кадра t:

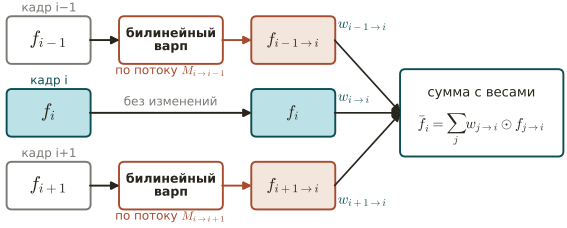
$$S_t(j) = s_t^j + \max_{i:\ \mathrm{IoU}(b_{t-1}^i,\, b_t^j) \geq 0.5} S_{t-1}(i), \qquad S_1(j) = s_1^j$$

Лучший путь заканчивается в $\arg\max_j S_T(j)$; сам путь восстанавливаем по запомненным arg max. Цена $O(T \cdot M^2)$.

Затем score боксов пути заменяем на среднее по пути (на рисунке $2.95/4 = 0.74$), пересекающиеся боксы подавляем и ищем следующий путь.

1. FGFA: агрегация признаков

Zhu et al., ICCV 2017



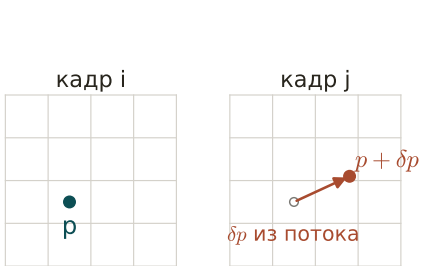
Flow-Guided Feature Aggregation

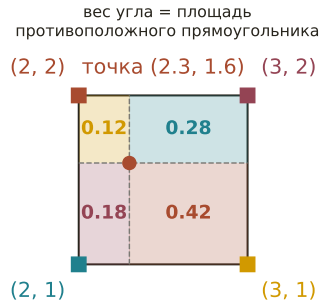
$$f_{j\to i} = \mathcal{W}\big(f_j,\ M_{i\to j}\big), \qquad \bar{f}_i = \sum_{j=i-K}^{i+K} w_{j\to i} \odot f_{j\to i}, \qquad \sum_j w_{j\to i} = 1$$

f_j — карта признаков кадра j; $M_{i\to j}$ — оптический поток (лекция 2, FlowNet); $\mathcal{W}$ — билинейный варп: перенос признаков соседа в координаты кадра i (следующий слайд). Вес $w_{j\to i}$ в каждом пикселе растёт с косинусной близостью $f_{j\to i}$ и f_i: если варп удался, соседу верим. $\odot$ — поэлементное умножение.

1. Билинейный варп: идея

Поток $M_{i\to j}$ говорит для каждого пикселя p кадра i: «в кадре j эта точка находится в $p + \delta p$». Смещение δp дробное, а признаки f_j заданы только в целых узлах сетки.





Берём 4 ближайших узла и усредняем с весами: чем ближе узел, тем больше вес. Точка $(2.3,\ 1.6)$: вес узла $(2, 1)$ равен $(1-0.3)(1-0.6) = 0.28$, и так далее: $f = 0.28\, f(2,1) + 0.12\, f(3,1) + 0.42\, f(2,2) + 0.18\, f(3,2)$; сумма весов 1.

1. Билинейный варп: формула

Варп признаков соседа в координаты кадра i

$$f_{j\to i}(p) = \sum_q G\big(q,\ p+\delta p\big)\, f_j(q)$$

$$G(q,p') = \max\big(0,\ 1-|q_x-p'_x|\big)\cdot \max\big(0,\ 1-|q_y-p'_y|\big)$$

p — пиксель кадра i; $\delta p = M_{i\to j}(p)$ — смещение из потока; q — целые узлы сетки кадра j; $p' = p+\delta p$. Множитель G по каждой оси: 1 в самом узле, 0 на расстоянии ≥ 1 — поэтому работают только 4 ближайших узла.

Почему не «ближайший пиксель»: G гладко зависит от δp, поэтому градиент доходит до сети потока — FGFA учит FlowNet вместе с детектором. В PyTorch это `grid_sample`.

1. Что дало время детектору

R-FCN, ResNet-101	mAP	медленные	быстрые
покадровый детектор	74.1	83.6	51.2
+ FGFA	77.1	85.9	56.1
+ FGFA + Seq-NMS	78.9	86.8	57.9

ImageNet VID; mAP отдельно для медленных и быстрых объектов; репозиторий msracver/Flow-Guided-Feature-Aggregation

- Выигрыш больше всего на быстрых объектах: $51.2 \to 57.9$ — там размытие сильнее всего.
- Цена: поток и варп для десятков соседних кадров на каждый кадр.

Сегодня обычно так: детектор покадровый и быстрый (YOLO26 — без NMS, январь 2026; RT-DETR), а временную согласованность обеспечивает трекер. Дальше вся лекция — про трекер.

1. Четыре парадигмы MOT: где живёт время

Tracking-by-detection время — в ассоциации	**Joint detection & tracking** одна сеть: боксы и связи	**Track queries (трансформер)** запрос = объект во времени	**Tracking-by-segmentation** маска + память SAM 2
– SORT 2016 – DeepSORT 2017 – ByteTrack 2021 – OC-SORT 2022 – TrackTrack 2025	– Tracktor 2019 – CenterTrack 2020 – FairMOT 2021	– MOTR 2022 – MOTRv2 2023 – MOTIP 2025	– SAM 2 2024 – SAM2MOT 2025 – → лекция 6

Tracking-by-detection (TbD) — детектор отдельно, связь кадров отдельно. До сих пор лидирует на MOT17 и стоит в продакшене, поэтому разделы 2–5 про него. Остальные колонки — в разделе 6 и в лекции 6.

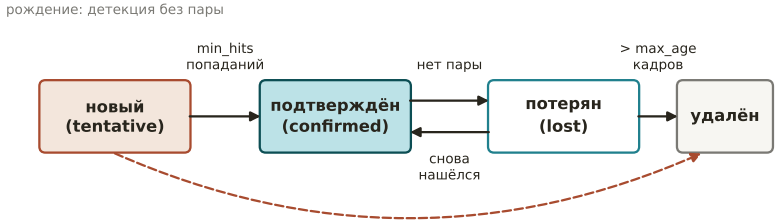
2. Tracking-by-detection: цикл на каждом кадре



1. **Прогноз**: где каждый трек должен быть *сейчас*, если он двигается как раньше.
2. **Стоимость**: насколько каждая детекция похожа на каждый трек.
3. **Назначение**: какой трек получает какую детекцию (каждый — не больше одной).
4. **Обновление**: уточнить треки по их детекциям; родить треки из лишних детекций; удалить давно пропавшие.

Детектор и трекер меняются независимо: лучше детектор — лучше треки, без переобучения трекера.

2. Жизненный цикл трека



Параметры жизненного цикла

min_hits — сколько кадров подряд трек должен получать детекцию, чтобы считаться настоящим (SORT: 3). Защищает от ложных боксов.

max_age — сколько кадров трек может прожить без детекции (SORT: 1; DeepSORT, ByteTrack: 30). Чем больше, тем лучше трек переживает окклюзии, но тем легче ему «украсть» чужую детекцию.

$\tau_{\textbf{new}}$ — минимальная уверенность детекции, из которой можно родить трек.

2. Листинг: один шаг трекера

Алгоритм: tracking-by-detection, кадр t

вход: треки $\mathcal{T}$ после кадра $t-1$; детекции $\mathcal{D}_t$
выход: треки после кадра t; подтверждённые — в ответ

1. для каждого трека i: прогноз $\hat{b}_i$ на кадр t (раздел 3)
2. матрица стоимостей $C \in \mathbb{R}^{N \times M}$: трек i против детекции j (раздел 4)
3. назначение: пары (i, j) с минимальной суммой C_{ij}; пары дороже порога τ разорвать
4. пары: обновить трек i по детекции j; hits$_i$+1; пропуск$_i \leftarrow 0$
5. треки без пары: пропуск$_i + 1$; если пропуск$_i >$ max_age — удалить
6. детекции без пары и $s \geq \tau_{\text{new}}$: новый трек (статус «новый»)
7. вернуть треки с hits $\geq$ min_hits

Шаги 1 и 4 — модель движения (фильтр Калмана). Шаги 2–3 — ассоциация. Их и разбираем дальше.

3. Зачем трекеру модель движения

В кадре t о положении объекта у нас два источника знания:

- **прогноз** по прошлому: «шёл вправо 3 px за кадр — значит, сейчас здесь»;
- **детекция** в кадре t: «детектор видит бокс здесь».

Оба неточные: объект мог ускориться, детектор ошибается на несколько пикселей.

Прогноз нужен трижды

Ассоциация: сравнивать детекцию кадра t надо с положением трека *в кадре* t, а не $t-1$.
Окклюзия: детекции нет — трек двигается по прогнозу и не теряется.
Сглаживание: дрожание бокса от кадра к кадру усредняется.

Вопрос раздела: *как правильно сложить две неточные оценки одного числа?* Ответ на него в 1D и есть фильтр Калмана; потом перепишем то же для векторов.

3. Две неточные оценки одного числа

Пусть x — истинная координата центра бокса. Есть две оценки:

$$a = x + e_a, \quad \mathbb{E}e_a = 0, \ \mathrm{Var}\, e_a = \sigma_a^2 \qquad\qquad z = x + e_z, \quad \mathbb{E}e_z = 0, \ \mathrm{Var}\, e_z = \sigma_z^2$$

a — прогноз, z — детекция; e_a, e_z — их ошибки, независимые; σ^2 — насколько каждая оценка обычно ошибается.

Ищем оценку как взвешенное среднее

$$\hat{x} = (1-k)\,a + k\,z \;=\; a + k\,(z-a)$$

$k \in [0,1]$ — доля доверия детектору; $z - a$ — *невязка*: насколько детекция разошлась с прогнозом. Вторая запись читается так: «берём прогноз и сдвигаем его к детекции на долю k расхождения».

Ошибка такой оценки

$\hat{x} - x = (1-k)e_a + k\,e_z \;\Rightarrow\; \sigma^2(k) = (1-k)^2\sigma_a^2 + k^2\sigma_z^2$ (перекрёстного члена нет, так как e_a, e_z независимы)

3. Лучший вес k: минимизируем дисперсию ошибки

$\sigma^2(k)$ — парабола по k. Приравниваем производную к нулю:

$$\frac{d\sigma^2}{dk} = -2(1-k)\,\sigma_a^2 + 2k\,\sigma_z^2 = 0 \quad \Longrightarrow \quad \boxed{k = \frac{\sigma_a^2}{\sigma_a^2 + \sigma_z^2}}$$

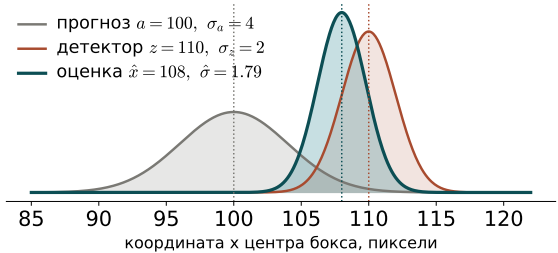
Подставляем обратно:

$$\hat{\sigma}^2 = (1-k)\,\sigma_a^2 = \frac{\sigma_a^2\,\sigma_z^2}{\sigma_a^2 + \sigma_z^2} \;<\; \min(\sigma_a^2, \sigma_z^2)$$

- Вес детектора тем больше, чем *хуже* прогноз (σ_a^2 велико) и чем *лучше* детектор (σ_z^2 мало).
- Оценка точнее *каждого* источника: две независимые ошибки частично гасят друг друга.

Это и есть коэффициент Калмана в 1D. Весь фильтр — эта формула плюс правило, как прогноз и его σ_a^2 получаются из прошлого.

3. Пример: прогноз 100 ± 4, детектор 110 ± 2

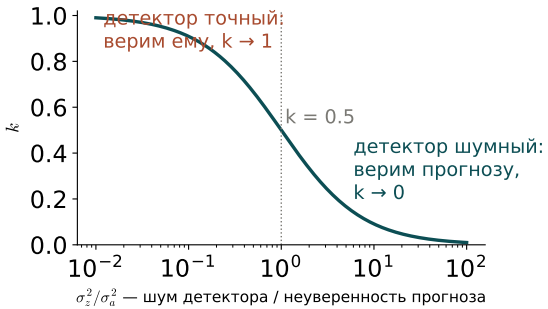


$$k = \frac{4^2}{4^2 + 2^2} = \frac{16}{20} = 0.8, \qquad \hat{x} = 100 + 0.8\,(110 - 100) = 108$$

$$\hat{\sigma}^2 = (1 - 0.8) \cdot 16 = 3.2, \qquad \hat{\sigma} \approx 1.79$$

Оценка ближе к более точному источнику (детектору), а её разброс 1.79 меньше, чем у обоих (4 и 2). Гауссианы на рисунке — распределения ошибок, ширина = σ.

3. Предельные случаи веса k



Перепишем $k = \dfrac{1}{1 + \sigma_z^2/\sigma_a^2}$: вес зависит только от отношения шумов.

- $\sigma_z \to 0$ (идеальный детектор): $k \to 1$, $\hat{x} \to z$.
- $\sigma_z \to \infty$ или детекции нет: $k \to 0$, $\hat{x} = a$ — живём на прогнозе.
- $\sigma_a \to 0$ (прогноз точен): $k \to 0$, детекцию почти игнорируем.

Вопрос аудитории: куда в этой картинке ложится детекция с низким score? (Её σ_z больше — вес должен быть меньше. Так рассуждают адаптивные варианты Калмана, например NSA в StrongSORT.)

3. Добавляем время: шаг предсказания

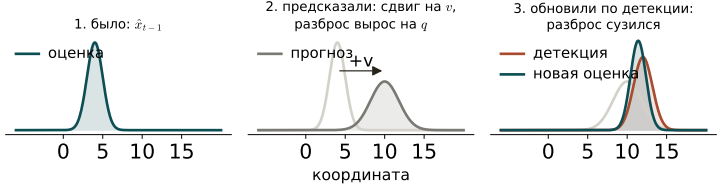
Модель «постоянная скорость» в 1D: за кадр координата сдвигается на скорость v, скорость почти не меняется:

$$x_t = x_{t-1} + v\,\Delta t + \text{шум}, \qquad \Delta t = 1 \text{ кадр}$$

Прогноз и его неопределённость

$$a_t = \hat{x}_{t-1} + v\,\Delta t, \qquad \sigma^2_{a,t} = \hat{\sigma}^2_{t-1} + q$$

$\hat{x}_{t-1}, \hat{\sigma}^2_{t-1}$ — оценка и её дисперсия после прошлого кадра; q — *шум процесса*: насколько реальное движение за один кадр отклоняется от равномерного. Каждый шаг без измерений добавляет q.



3. Весь фильтр в 1D: два шага на кадр

Предсказание (predict)

$a_t = \hat{x}_{t-1} + v$

$\sigma^2_{a,t} = \hat{\sigma}^2_{t-1} + q$

сдвинули оценку по модели движения; неуверенность выросла

Обновление (update)

$k_t = \sigma^2_{a,t} \,/\, (\sigma^2_{a,t} + \sigma^2_z)$

$\hat{x}_t = a_t + k_t\,(z_t - a_t)$

$\hat{\sigma}^2_t = (1 - k_t)\,\sigma^2_{a,t}$

сложили прогноз с детекцией; неуверенность упала

- Детекции нет (окклюзия) — делаем только predict: $\hat{x}_t = a_t$, $\hat{\sigma}^2_t = \sigma^2_{a,t}$. С каждым кадром σ^2 растёт на q.
- Скорость v тоже неизвестна — её надо оценивать вместе с координатой. Значит, состояние — *вектор* (x, v), и нужна матричная запись.

Дальше ничего нового по смыслу: те же две строки, где числа заменены векторами, а дисперсии — ковариационными матрицами.

3. Откуда берутся дисперсии

В цикле три дисперсии, но задавать руками нужно только две.

$\hat{\sigma}^2_{t-1}$ — считает сам фильтр

Это результат прошлого update. В момент рождения трека: $\hat{x}_0 = z_0$, $\hat{\sigma}^2_0 = \sigma^2_z$ — координату знаем с точностью детектора; дисперсию скорости ставят большой — скорость пока неизвестна.

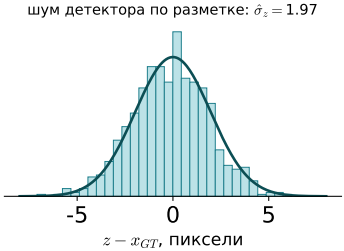
σ^2_z — шум детектора: по размеченным данным

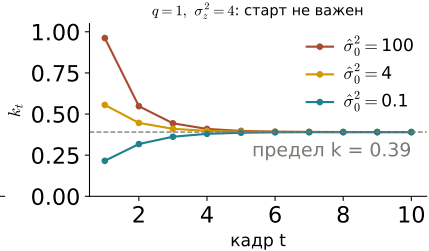
Сопоставляем детекции с разметкой (IoU ≥ 0.5), считаем ошибки $e_n = z_n - x^{GT}_n$ и их выборочную дисперсию: $\hat{\sigma}^2_z = \frac{1}{N}\sum_{n=1}^{N}(e_n - \bar{e})^2$, где $\bar{e}$ — средняя ошибка, N — число пар. DeepSORT вместо этого берёт эвристику $\sigma_z = h/20$.

q — шум движения: по размеченным трекам или подбором

Насколько реальный шаг отклоняется от модели: $\hat{q} = \mathrm{Var}\big(x^{GT}_t - x^{GT}_{t-1} - \bar{v}\big)$, $\bar{v}$ — средняя скорость трека. На практике q (и σ^2_z) часто просто подбирают по HOTA/IDF1 на валидации.

3. Старт не важен, важно отношение q/σ_z^2





Установившийся режим

В пределе $s = \sigma_{a,t}^2$ одинаково во всех кадрах: $s = \frac{s\,\sigma_z^2}{s+\sigma_z^2} + q$, откуда $s = \frac{q+\sqrt{q^2+4q\sigma_z^2}}{2}$. При $q = 1,\ \sigma_z^2 = 4$: $s \approx 2.56$, $k = \frac{2.56}{2.56+4} \approx 0.39$.

k зависит только от q/σ_z^2. Больше q — больше веры детектору; меньше q — глаже, но трек хуже успевает за поворотом.

3. От числа к вектору: состояние трека

Состояние трека в DeepSORT и ByteTrack

$$\mathbf{x} = \left[x,\ y,\ a,\ h,\ \dot{x},\ \dot{y},\ \dot{a},\ \dot{h} \right]^\top \in \mathbb{R}^8$$

(x, y) — центр бокса; $a = w/h$ — отношение сторон; h — высота; точка сверху — скорость этой величины (изменение за кадр).

Неопределённость — ковариационная матрица

$P = \mathrm{Cov}(\mathbf{x} - \hat{\mathbf{x}}) \in \mathbb{R}^{8\times 8}$. На диагонали — дисперсии каждой компоненты (обобщение σ^2), вне диагонали — насколько ошибки компонент связаны. Например, если ошиблись в скорости $\dot{x}$, то ошибёмся и в будущем x: $P_{x\dot{x}} \neq 0$.

Варианты: SORT — $[u, v, s, r, \dot{u}, \dot{v}, \dot{s}]$ с площадью s и постоянным $r = w/h$; BoT-SORT — $[x, y, w, h, \dots]$: ширина предсказывается напрямую, боксы точнее.

3. Модель движения: матрица F

Постоянная скорость для всех четырёх величин сразу:

$$\mathbf{x}_t = F\,\mathbf{x}_{t-1} + \mathbf{w}_t, \qquad F = \begin{pmatrix} I_4 & \Delta t\, I_4 \\ 0 & I_4 \end{pmatrix}, \qquad \mathbf{w}_t \sim \mathcal{N}(0, Q)$$

- I_4 — единичная 4×4; верхняя строка блоков: $x_t = x_{t-1} + \Delta t\, \dot{x}_{t-1}$ (и так же для y, a, h); нижняя: $\dot{x}_t = \dot{x}_{t-1}$.
- $\mathbf{w}_t$ — шум процесса, Q — его ковариация (матричный аналог q).

Проверка: строка для x

$$\big(F\mathbf{x}\big)_1 = 1 \cdot x + 0 \cdot y + 0 \cdot a + 0 \cdot h + \Delta t \cdot \dot{x} + 0 + 0 + 0 = x + \Delta t\, \dot{x}$$

Модель линейная: F не зависит от состояния. Именно это позволяет фильтру Калмана быть точным и дешёвым — и именно это ломается при разворотах (слайд в конце раздела).

3. Измерение: матрица H и шумы Q, R

Что видит детектор

$$\mathbf{z}_t = H\,\mathbf{x}_t + \mathbf{v}_t, \qquad H = \begin{pmatrix} I_4 & 0_{4\times 4} \end{pmatrix}, \qquad \mathbf{v}_t \sim \mathcal{N}(0, R)$$

$\mathbf{z}_t = (x, y, a, h)$ — бокс детектора. H «выбирает» из состояния первые 4 компоненты: скорости детектор не измеряет. R — ковариация ошибки детектора (аналог σ_z^2).

Как задают шумы (DeepSORT, ByteTrack)

Диагональные, стандартные отклонения пропорциональны высоте бокса h:

$$\sigma_{\text{поз}} = \tfrac{1}{20} h, \qquad \sigma_{\text{скор}} = \tfrac{1}{160} h$$

Q: позиции — $\sigma^2_{\text{поз}}$, скорости — $\sigma^2_{\text{скор}}$; R: позиции — $\sigma^2_{\text{поз}}$. Для a — малые константы.

Масштаб по h: далёкий маленький человек дрожит на 1–2 px, близкий большой — на 10.

3. Предсказание в матричной форме

Обозначение: $\hat{\mathbf{x}}_{t|t-1}$ — оценка для кадра t по данным до кадра $t-1$ (прогноз), $\hat{\mathbf{x}}_{t|t}$ — после учёта детекции кадра t.

Predict

$$\hat{\mathbf{x}}_{t|t-1} = F\,\hat{\mathbf{x}}_{t-1|t-1}, \qquad P_{t|t-1} = F\,P_{t-1|t-1}\,F^{\top} + Q$$

1D: $a_t = \hat{x}_{t-1} + v$ и $\sigma_a^2 = \hat{\sigma}^2 + q$. Откуда $FPF^{\top}$: если ошибка $\mathbf{e}$ имеет ковариацию P, то у $F\mathbf{e}$ она $FPF^{\top}$ (свойство ковариации линейного преобразования).

Что это даёт: координата и скорость, $\Delta t = 1$

$$P = \begin{pmatrix} \sigma_x^2 & 0 \\ 0 & \sigma_{\dot{x}}^2 \end{pmatrix} \;\Rightarrow\; FPF^{\top} = \begin{pmatrix} \sigma_x^2 + \sigma_{\dot{x}}^2 & \sigma_{\dot{x}}^2 \\ \sigma_{\dot{x}}^2 & \sigma_{\dot{x}}^2 \end{pmatrix}$$

Неуверенность в скорости перешла в неуверенность в положении, и появилась связь «ошибка скорости $\rightarrow$ ошибка положения».

3. Обновление: та же формула, что в 1D

Смысл	1D	Матрицы
прогноз в пространстве детектора	a_t	$H\hat{\mathbf{x}}_{t\|t-1}$
невязка (инновация)	$z_t - a_t$	$\mathbf{y}_t = \mathbf{z}_t - H\hat{\mathbf{x}}_{t\|t-1}$
разброс невязки	$\sigma_a^2 + \sigma_z^2$	$S_t = HP_{t\|t-1}H^\top + R$
коэффициент	$k = \sigma_a^2/(\sigma_a^2 + \sigma_z^2)$	$K_t = P_{t\|t-1}H^\top S_t^{-1}$
новая оценка	$a_t + k(z_t - a_t)$	$\hat{\mathbf{x}}_{t\|t} = \hat{\mathbf{x}}_{t\|t-1} + K_t\mathbf{y}_t$
новая неуверенность	$(1-k)\sigma_a^2$	$P_{t\|t} = (I - K_tH)P_{t\|t-1}$

K_t — матрица 8×4: переводит невязку бокса (4 числа) в поправку всего состояния (8 чисел). Так корректируется и *скорость*, которую детектор не видит: через связь $P_{x\dot{x}}$ в $P_{t|t-1}$. S_t ещё понадобится в ассоциации.

3. Откуда K: вывод на доске (1/2)

Ищем K, при котором ошибка после обновления минимальна. Обозначим $\mathbf{e}^- = \mathbf{x} - \hat{\mathbf{x}}_{t|t-1}$ (ошибка прогноза, ковариация P^-) и $\mathbf{e} = \mathbf{x} - \hat{\mathbf{x}}_{t|t}$.

Шаг 1: выразить ошибку через K

Подставляем $\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + K(\mathbf{z} - H\hat{\mathbf{x}}_{t|t-1})$ и $\mathbf{z} = H\mathbf{x} + \mathbf{v}$:

$$\mathbf{e} = \mathbf{e}^- - K\big(H\mathbf{e}^- + \mathbf{v}\big) = (I - KH)\,\mathbf{e}^- - K\,\mathbf{v}$$

Шаг 2: ковариация ошибки (форма Джозефа)

$\mathbf{e}^-$ и $\mathbf{v}$ независимы, поэтому ковариации двух слагаемых складываются:

$$P = (I - KH)\,P^-\,(I - KH)^\top + K\,R\,K^\top$$

Верна для *любого* K. 1D: $(1-k)^2\sigma_a^2 + k^2\sigma_z^2$ — ровно наша парабола.

3. Откуда K: вывод на доске (2/2)

Шаг 3: минимизируем след

След $\operatorname{tr} P = \sum_i P_{ii}$ — сумма дисперсий всех компонент, «общий разброс». Раскрываем скобки и дифференцируем по K, используя $\frac{\partial}{\partial K}\operatorname{tr}(KA) = A^\top$ и $\frac{\partial}{\partial K}\operatorname{tr}(KCK^\top) = 2KC$ для симметричной C:

$$\frac{\partial \operatorname{tr} P}{\partial K} = -2\,P^- H^\top + 2K\big(HP^- H^\top + R\big) = 0$$

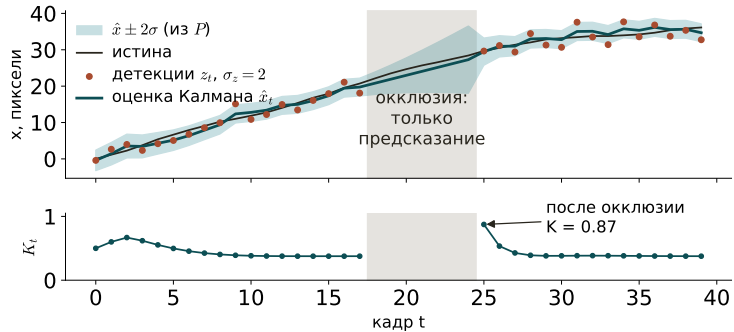
Шаг 4: решение и подстановка

$$K = P^- H^\top \big(HP^- H^\top + R\big)^{-1} = P^- H^\top S^{-1}, \qquad P = (I - KH)\,P^-$$

Вторая формула — подстановка оптимального K в форму Джозефа (слагаемые сокращаются).

При линейной модели и гауссовых шумах это лучшая оценка в смысле средней квадратичной ошибки. Заметьте: P и K не зависят от самих детекций — только от F, H, Q, R.

3. Калман на треке с окклюзией



- Без окклюзии K_t быстро выходит на ≈ 0.4: оценка гладче красных точек детектора.
- В окклюзии только predict: коридор $\pm 2\sigma$ расширяется на Q за кадр.
- Первая детекция после окклюзии: $K = 0.87$ — прогнозу уже мало верим, оценка прыгает к детекции.

3. Листинг: фильтр Калмана для бокса

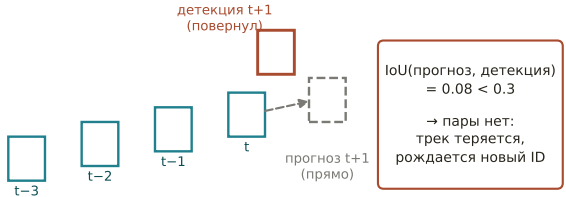
Алгоритм: Kalman filter, постоянная скорость

вход: детекции трека $\mathbf{z}_1, \mathbf{z}_2, \ldots$ (или «нет детекции»); F, H, Q, R
выход: оценки $\hat{\mathbf{x}}_t$ и неопределённости P_t

1. инициализация по первой детекции: $\hat{\mathbf{x}} = [\mathbf{z}_1,\ \mathbf{0}]$; P — большие дисперсии скоростей
2. **для** каждого кадра t:
   - 2.1 predict: $\hat{\mathbf{x}} \leftarrow F\hat{\mathbf{x}}$, $P \leftarrow FPF^\top + Q$
   - 2.2 **если** есть детекция $\mathbf{z}_t$: $S = HPH^\top + R$, $K = PH^\top S^{-1}$
   - 2.3 $\hat{\mathbf{x}} \leftarrow \hat{\mathbf{x}} + K(\mathbf{z}_t - H\hat{\mathbf{x}})$, $P \leftarrow (I - KH)P$

Самая дорогая операция — обратить S размера 4×4. Поэтому SORT обрабатывает сотни кадров в секунду на CPU: время уходит на детектор, а не на трекер.

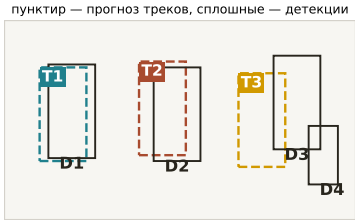
3. Где постоянная скорость ломается



- Разворот, прыжок, танец, спорт: реальное движение не равномерное, прогноз уходит в сторону.
- В окклюзии хуже: прогноз двигается по старой скорости много кадров, ошибка растёт примерно как $\Delta v \cdot (\text{длина окклюзии})$.
- Если IoU прогноза с детекцией падает ниже порога, пары нет — трек теряется, рождается новый ID.

Главный провал Калмана в MOT. Ответы в разделе 5: OC-SORT (опора на наблюдения) и BoT-SORT (вычесть движение камеры).

4. Ассоциация: кто есть кто



$C_{ij} = 1 - \mathrm{IoU}(\hat{b}_i, d_j)$

	D1	D2	D3	D4
T1	0.12	0.85	0.95	0.99
T2	0.90	0.20	0.70	0.97
T3	0.93	0.55	0.35	0.98

Матрица стоимостей

N треков (после predict), M детекций кадра t. $C \in \mathbb{R}^{N \times M}$, C_{ij} — «цена» решения «детекция j — продолжение трека i»: чем меньше, тем вероятнее.

Три вопроса раздела: **чем** измерять C_{ij}; **как** выбрать пары по всей матрице; **когда** отказаться от пары.

4. Стоимость 1: геометрия через IoU

IoU-стоимость (SORT, ByteTrack, OC-SORT)

$$C_{ij}^{\text{IoU}} = 1 - \text{IoU}\big(\hat{b}_i,\ d_j\big), \qquad \hat{b}_i = H\hat{\mathbf{x}}_{i,\,t|t-1}$$

$\hat{b}_i$ — бокс трека i, предсказанный Калманом на кадр t; d_j — бокс детекции j. Полное совпадение даёт 0, отсутствие пересечения — 1.

крупный медленный: IoU = 0.71



мелкий быстрый: IoU = 0



Плюсы

- дёшево, без обучения
- естественно работает с размером бокса

Минусы

- мелкий быстрый объект: IoU $= 0$ уже при небольшом сдвиге
- не учитывает, насколько трек уверен в прогнозе

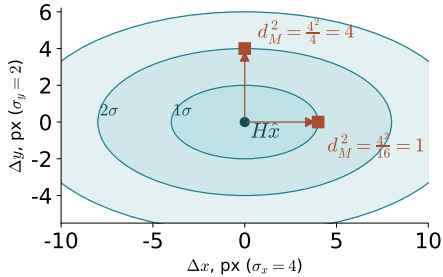
4. Стоимость 2: расстояние Махаланобиса

В 1D мера — «на сколько сигм»: $(z-a)^2/\sigma^2$. 4 px — много при $\sigma=1$, мало при $\sigma=10$.

Махаланобис (DeepSORT)

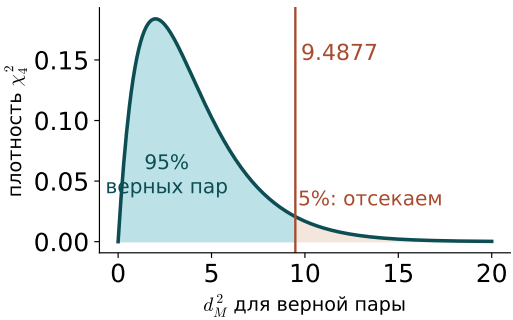
$$d_M^2(i,j) = \mathbf{y}_{ij}^\top S_i^{-1} \mathbf{y}_{ij}, \qquad \mathbf{y}_{ij} = d_j - H\hat{\mathbf{x}}_i$$

$\mathbf{y}_{ij}$ — невязка детекции j от прогноза трека i; S_i — её ковариация из update Калмана. Диагональная S: $d_M^2 = \sum_k y_k^2/S_{kk}$ — сумма квадратов «сигм».



Обе детекции на рисунке в 4 px от прогноза, но по x трек неуверен ($\sigma_x = 4$) — $d_M^2 = 1$, а по y уверен ($\sigma_y = 2$) — $d_M^2 = 4$.

4. Гейт: какие пары вообще возможны



Если детекция j действительно принадлежит треку i, то $\mathbf{y}_{ij} \sim \mathcal{N}(0, S_i)$. После «отбеливания» $S_i^{-1/2}\mathbf{y}$ это 4 независимые стандартные нормальные величины, а сумма их квадратов распределена по χ^2 с 4 степенями свободы:

$$d^2_M \sim \chi^2_4, \qquad \Pr\left(d^2_M \le 9.4877\right) = 0.95$$

Gating

$C_{ij} = +\infty$, если $d^2_M(i,j) > 9.4877$: пара невозможна, даже если других кандидатов нет. 4 — размерность бокса (x, y, a, h); 0.95 — доля верных пар, которую гейт пропускает.

4. Стоимость 3: внешность (ReID)



ReID-сеть: вырезка бокса → вектор (128–512 чисел)
сравниваем r_j с каждым $r^{(k)}$ по косинусу
и берём самый похожий

Косинусная стоимость и смесь стоимостей

$$C_{ij}^{\text{app}} = 1 - \max_k \frac{r_j^\top r_i^{(k)}}{\|r_j\| \, \|r_i^{(k)}\|}, \qquad C_{ij} = \lambda \, d_M^2(i,j) + (1-\lambda) \, C_{ij}^{\text{app}}$$

r_j — эмбеддинг вырезки детекции j (ReID-сеть, обученная различать людей); $r_i^{(k)}$ — k-й из последних эмбеддингов трека i (в DeepSORT до 100); дробь — косинус угла между векторами; λ — вес геометрии.

В DeepSORT $\lambda = 0$: Махаланобис — только гейт. Внешность спасает после долгой окклюзии и бесполезна, когда все одинаковые (танцы, спорт).

4. Задача о назначениях

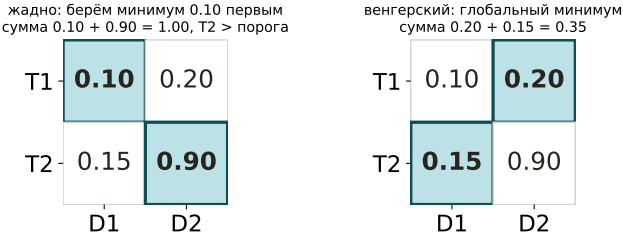
Линейная задача о назначениях

$$X^* = \arg\min_{X \in \{0,1\}^{N \times M}} \sum_{i=1}^{N} \sum_{j=1}^{M} C_{ij} X_{ij} \quad \text{при} \quad \sum_{j} X_{ij} \le 1, \quad \sum_{i} X_{ij} \le 1$$

$X_{ij} = 1$ — трек i получает детекцию j. Ограничения: у трека не больше одной детекции и у детекции не больше одного трека. Минимизируем суммарную стоимость *всех* пар сразу.

- Для квадратной матрицы $n \times n$ это выбор перестановки: $n!$ вариантов. При 50 треках перебор невозможен.
- Задача имеет точное решение за $O(n^3)$ — венгерский алгоритм (Kuhn 1955, Munkres 1957).
- Прямоугольную $N \times M$ дополняют до квадрата фиктивными строками или столбцами с большой стоимостью.

4. Почему не жадно



- Жадный алгоритм: берём самую дешёвую пару, вычёркиваем строку и столбец, повторяем.
- Пример: T1–D1 (0.10) выглядит лучшей парой, но после неё T2 достаётся D2 за 0.90 — выше порога, T2 остаётся без пары.
- Оптимум: T1–D2 + T2–D1 $= 0.35$. Каждая пара чуть хуже, но в сумме намного лучше.

Такие конфликты бывают, когда две детекции близки к двум трекам одновременно — то есть ровно в толпе и на пересечениях, где ошибки дороже всего.

4. Венгерский алгоритм: идея

Наблюдение 1: сдвиг строки не меняет оптимум

Вычтем число u_i из всех элементов строки i. В любом полном назначении из строки i берётся ровно один элемент, поэтому стоимость *каждого* назначения уменьшится на одно и то же u_i. Порядок назначений по стоимости не изменится. То же для столбцов.

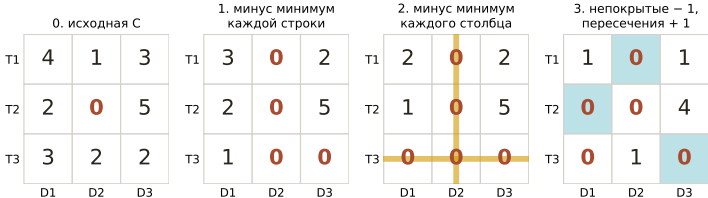
Наблюдение 2: назначение по нулям оптимально

Если все элементы ≥ 0 и нашлось назначение только по нулевым клеткам, его стоимость 0 — меньше быть не может.

Наблюдение 3: когда нулей не хватает

Теорема Кёнига: наибольшее число нулей, никакие два из которых не лежат в одной строке или столбце, равно наименьшему числу линий, покрывающих все нули. Пока линий меньше n, полного назначения по нулям нет — нужно создать новые нули.

4. Венгерский алгоритм на примере 3×3



1. Из каждой строки вычли её минимум (1, 0, 2); из каждого столбца — его минимум (1, 0, 0).
2. Все нули покрываются двумя линиями (столбец D2 и строка T3): $2 < 3$, назначения по нулям нет.
3. Наименьший непокрытый элемент 1: вычли его из непокрытых, прибавили к пересечению линий.
4. Три независимых нуля: T1–D2, T2–D1, T3–D3. Стоимость по исходной матрице $1 + 2 + 2 = 5$; перебор всех $3! = 6$ перестановок даёт 6, 11, **5**, 9, 7, 6.

4. Листинг: венгерский алгоритм

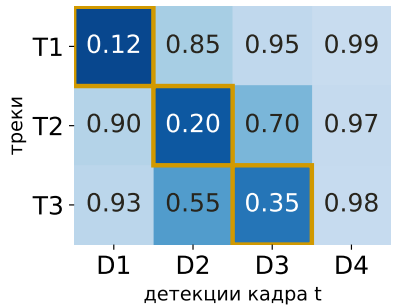
Алгоритм: Hungarian (Kuhn–Munkres)

вход: матрица $C \in \mathbb{R}^{n \times n}$
выход: перестановка σ с минимальной $\sum_i C_{i,\sigma(i)}$

1. из каждой строки вычесть её минимум; затем из каждого столбца — его минимум
2. покрыть все нули наименьшим числом горизонтальных и вертикальных линий
3. **пока** линий меньше n:
   - 3.1 $m \leftarrow$ наименьший непокрытый элемент
   - 3.2 вычесть m из непокрытых, прибавить m на пересечениях линий; покрыть нули заново
4. выбрать n нулей, по одному в каждой строке и каждом столбце

$O(n^3)$, для 100 треков — доли миллисекунды. На практике — `scipy.optimize.linear_sum_assignment` (работает и с прямоугольной C).

4. После назначения — порог



Венгерский алгоритм назначит пару каждому, даже если пара плохая. Поэтому пары проверяют порогом:

$$\text{пары} = \left\{(i,j):\ X^*_{ij} = 1,\ C_{ij} \le \tau\right\}$$

$\tau = 1 - \text{IoU}_{\min}$; в SORT $\text{IoU}_{\min} = 0.3$, то есть $\tau = 0.7$.

- Здесь назначены T1–D1, T2–D2, T3–D3 (сумма 0.67 — минимум из 24 вариантов).
- D4 ни с кем не пересекается: детекция без пары $\rightarrow$ кандидат в новый трек.
- Трек без пары $\rightarrow$ пропуск: живёт на прогнозе до max_age кадров.

5. SORT: минимальный трекер

Bewley et al., ICIP 2016

SORT = Simple Online and Realtime Tracking

Ровно цикл из раздела 2: Калман с постоянной скоростью + стоимость $1 - \text{IoU}$ + венгерский алгоритм + порог.

Состояние $[u, v, s, r, \dot{u}, \dot{v}, \dot{s}]$: центр (u, v), площадь s, отношение сторон r (без скорости — считается постоянным).

$\text{IoU}_{\text{min}} = 0.3$; min_hits $= 3$; max_age $= 1$; детектор Faster R-CNN; 260 кадров/с на CPU.

Плюсы

- простой: 200 строк кода
- очень быстрый
- качество почти целиком определяет детектор

Минусы

- max_age $= 1$: любая окклюзия — новый ID
- нет внешности: перепутанные на пересечении треки не расцепить

Все трекеры дальше — SORT, в котором заменили один блок. Удобно запоминать «какой провал лечит каждая идея».

5. DeepSORT: внешность и каскад

Wojke et al., ICIP 2017

Что поменяли относительно SORT

Состояние $[x, y, a, h, \dots] \in \mathbb{R}^8$ (раздел 3). **max_age** $= 30$: трек переживает окклюзию до секунды.

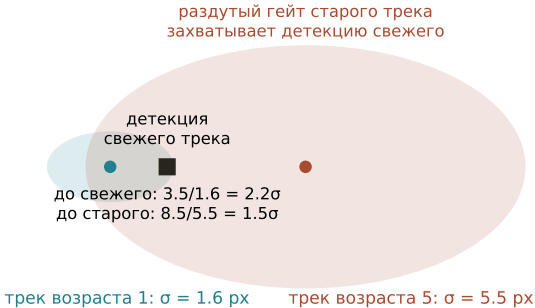
Внешность: ReID-сеть даёт 128-мерный эмбеддинг вырезки; у трека банк до 100 последних эмбеддингов; стоимость — косинусная.

Гейт: Махаланобис $d_M^2 \leq 9.4877$ и порог по косинусу.

Каскад по возрасту: сначала сопоставляются треки, пропустившие 0 кадров, потом 1, 2, ...

Итог в статье: число смен ID на MOT16 меньше на 45% относительно SORT. Цена — прогон ReID-сети по каждой детекции.

5. Зачем каскад по возрасту

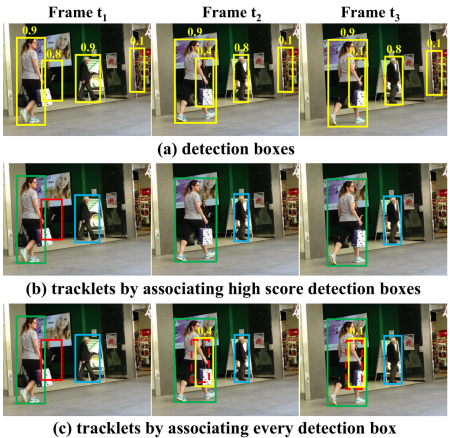


- Каждый кадр без детекции добавляет к P ещё Q, и $S = HPH^\top + R$ растёт.
- В $d_M^2 = \mathbf{y}^\top S^{-1}\mathbf{y}$ большая S делает расстояние *меньше*: старый трек «близок» ко всем детекциям вокруг.

Каскад: сначала отдать детекции свежим трекам, их прогноз точнее; старым — только то, что осталось.

5. ByteTrack: слабые боксы

Zhang et al., 2022



Zhang et al., ECCV 2022, Fig. 2

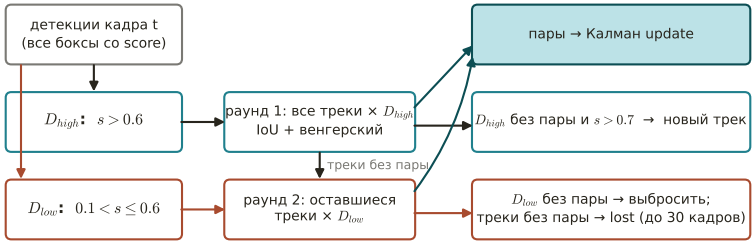
(a) детекции с уверенностью; (b) треки только из боксов с $s > 0.5$ — закрытый человек теряется; (c) ассоциация всех боксов сохраняет ID.

Наблюдение

Порог по score выбрасывает не только фон, но и *частично закрытые* объекты. У них уже есть трек, который их «ждёт».

Низкий score опасен для *рождения* трека (вдруг это фон), но полезен для *продолжения* уже существующего.

5. ByteTrack: два раунда сопоставления



- Детекции делятся по уверенности: D_{high} ($s > 0.6$) и D_{low} ($0.1 < s \leq 0.6$).
- Раунд 1: все треки против D_{high}. Раунд 2: треки, оставшиеся без пары, против D_{low} — только по IoU, внешность низкоуверенных боксов ненадёжна.
- Новые треки рождаются только из D_{high} с $s > 0.7$; потерянные треки ждут до 30 кадров.

MOT17 test: MOTA 80.3, IDF1 77.3, HOTA 63.1 при 30 кадрах/с на V100. ReID нет вообще — только Калман и IoU.

5. Листинг: ByteTrack

Алгоритм: BYTE, кадр t

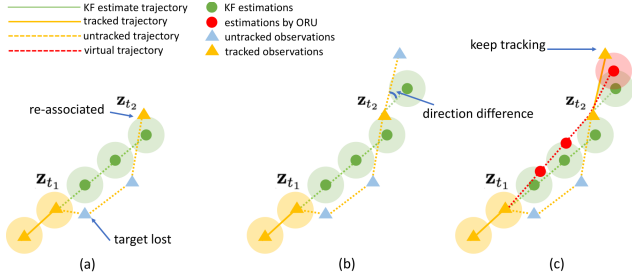
вход: треки $\mathcal{T}$; детекции $\mathcal{D}_t$; $\tau_{high} = 0.6$, $\tau_{low} = 0.1$, $\tau_{new} = 0.7$
выход: обновлённые треки

1. $D_{high} \leftarrow \{d : s > \tau_{high}\}$, $D_{low} \leftarrow \{d : \tau_{low} < s \leq \tau_{high}\}$
2. predict Калмана для всех треков $\mathcal{T}$
3. раунд 1: венгерский по $1 - \text{IoU}$ между $\mathcal{T}$ и D_{high}; пары $\rightarrow$ update
4. $\mathcal{T}_{rem} \leftarrow$ треки без пары; $D_{rem} \leftarrow$ детекции D_{high} без пары
5. раунд 2: венгерский между $\mathcal{T}_{rem}$ и D_{low}; пары $\rightarrow$ update
6. треки без пары во втором раунде $\rightarrow$ lost; удалить, если потеряны дольше 30 кадров
7. $d \in D_{rem}$ с $s > \tau_{new} \rightarrow$ новый трек; несопоставленные D_{low} выбросить

Почти все трекеры после 2021 года — ByteTrack плюс одна идея.

5. OC-SORT: проблема окклюзии

Cao et al., 2023



Cao et al., Observation-Centric SORT, Fig. 7

- (a) Трек потерян в t_1, найден снова в t_2. Всё это время Калман только предсказывал (зелёные точки).
- (b) Одна детекция $\mathbf{z}_{t_2}$ исправляет положение, но не направление: скорость всё ещё старая, на следующем кадре пара снова теряется.
- (c) OC-SORT пересчитывает фильтр по *виртуальным* наблюдениям (красные) — направление исправлено.

5. OC-SORT: виртуальная траектория (ORU)

ORU = Observation-centric Re-Update

Когда потерянный трек снова получил детекцию $\mathbf{z}_{t_2}$, между последним наблюдением $\mathbf{z}_{t_1}$ и новым строим прямую:

$$\tilde{\mathbf{z}}_t = \mathbf{z}_{t_1} + \frac{t - t_1}{t_2 - t_1}\left(\mathbf{z}_{t_2} - \mathbf{z}_{t_1}\right), \qquad t_1 < t < t_2$$

Дробь — доля пройденного пути к моменту t: 0 в t_1, 1 в t_2. Затем прогоняем predict и update Калмана по кадрам $t_1..t_2$, подставляя $\tilde{\mathbf{z}}_t$ вместо отсутствующих детекций.

Интуиция. Наблюдениям верим больше, чем прогнозу: «задним числом» считаем, что объект был виден и шёл по прямой — скорость в фильтре становится правильной.

Итог OC-SORT без ReID: HOTA 63.2 на MOT17 и 55.1 на DanceTrack, 793 кадра/с на CPU без детектора.

5. BoT-SORT: движение камеры Aharon et al., 2022



Компенсация движения камеры (CMC)

По точкам фона двух кадров оценивают аффинное преобразование $A = (M \mid T) \in \mathbb{R}^{2\times 3}$: поворот и масштаб $M \in \mathbb{R}^{2\times 2}$, сдвиг $T \in \mathbb{R}^2$ (RANSAC отбрасывает точки на людях). Прогноз каждого трека поправляют:

$$\hat{\mathbf{x}}' = \tilde{M}\hat{\mathbf{x}} + \tilde{T}, \qquad P' = \tilde{M}P\tilde{M}^\top, \qquad \tilde{M} = \mathrm{diag}(M, M, M, M)$$

$\tilde{M}$ применяет M к каждой паре компонент состояния;
$\tilde{T} = (T_1, T_2, 0, \ldots, 0)^\top$ сдвигает только центр.

Ещё: состояние $[x, y, w, h, \ldots]$, смесь IoU и ReID. MOT17 HOTA 65.0 (с ReID).

5. TrackTrack Shim et al., CVPR 2025

Track-Perspective-Based Association (TPA)

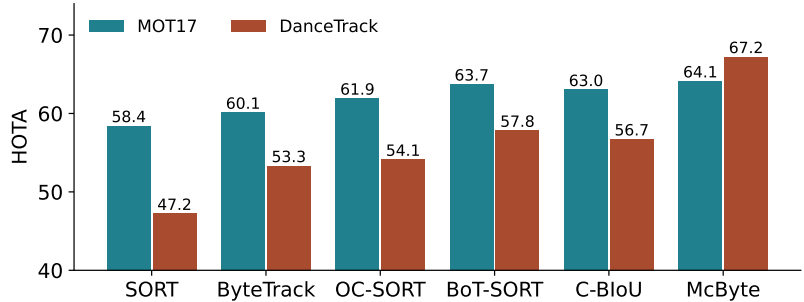
Вместо одного венгерского прогона по всей матрице — итеративно: пара (i, j) принимается, если детекция j самая дешёвая для трека i *и* трек i самый дешёвый для детекции j (взаимный минимум). В кандидаты идут боксы высокой и низкой уверенности, а также боксы, удалённые NMS детектора.

Track-Aware Initialization (TAI)

Новый трек рождается из детекции, только если она не перекрывается с уже сопоставленными треками: их боксы участвуют в NMS как «неудаляемые якоря».

MOT17 test: HOTA 67.1, IDF1 83.1, MOTA 81.8; MOT20: HOTA 65.7; DanceTrack: 66.5. Трекер по умолчанию в Ultralytics начиная с версии 8.4.63.

5. Один детектор — разные трекеры



Roboflow trackers v2.3.0 (март 2026): детекции YOLOX фиксированы, параметры по умолчанию, test через Codabench

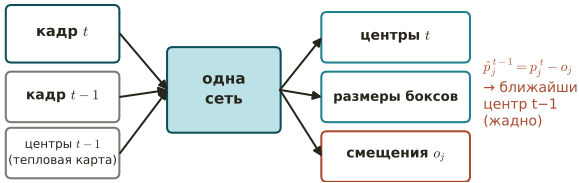
- Детектор одинаковый, поэтому разница — только ассоциация: до $+5$ HOTA на MOT17 и до $+20$ на DanceTrack.
- На DanceTrack (одинаковые костюмы, резкие движения) ассоциация сложнее детекции — там методы расходятся сильнее всего.

5. Сводка: каждая идея лечит один провал

Провал	Идея	Трекер
окклюзия рвёт трек	жить без детекции до 30 кадров	DeepSORT
перепутали после пересечения	эмбеддинг внешности	DeepSORT
старый трек ворует детекции	каскад по возрасту	DeepSORT
закрытые объекты с низким score	второй раунд по D_{low}	ByteTrack
скорость испорчена окклюзией	виртуальная траектория по наблюдениям	OC-SORT
движется камера	аффинная компенсация прогноза	BoT-SORT
дубли и лишние рождения	взаимный минимум, TAI	TrackTrack

На практике: ByteTrack или BoT-SORT поверх сильного детектора, затем менять один блок под домен.

6. Одна сеть даёт и боксы, и связи



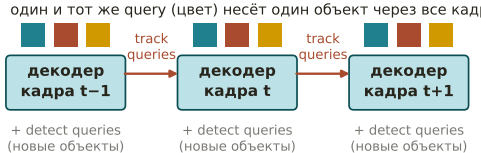
CenterTrack (Zhou et al., ECCV 2020)

Вход — кадры t, $t-1$ и карта центров объектов кадра $t-1$. Выход — центры p_j^t, размеры и смещение o_j каждого центра к прошлому кадру. Ассоциация: $\hat{p}_j^{t-1} = p_j^t - o_j$, ближайший центр кадра $t-1$ — жадно. MOT17 HOTA 52.2.

- **Tracktor** (2019): регрессионная голова детектора сдвигает бокс прошлого кадра в текущий — трекер без модели движения.
- **FairMOT** (2021): детекция и ReID на одной карте признаков; HOTA 59.3.

6. Track queries: MOTR

Zeng et al., ECCV 2022

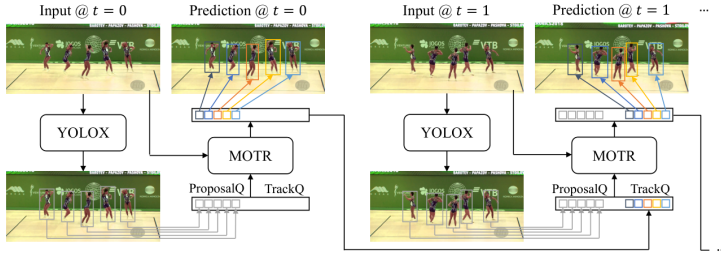


Как устроено

Основа — DETR (лекция 4, TadTR): набор запросов (queries), каждый декодируется в бокс. В MOTR выход запроса в кадре $t-1$ становится *track query* кадра t и продолжает «свой» объект: ID = номер запроса. Detect queries ищут новые объекты и сопоставляются с новыми GT венгерским алгоритмом. Учат на клипах: лосс усредняется по кадрам.

Track query — выученный аналог состояния Калмана и эмбеддинга ReID одновременно. MOT17 HOTA 57.8, DanceTrack 54.2: ассоциация хорошая, детекция слабее, чем у YOLOX.

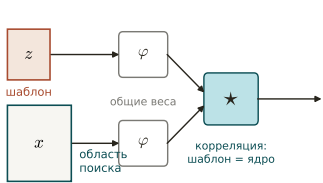
6. MOTRv2 и MOTIP 2023–2025



Zhang et al., MOTRv2, CVPR 2023, Fig. 2

- Боксы готового детектора YOLOX становятся *proposal queries*: трансформеру больше не нужно учиться находить людей, только связывать.
- MOT17 HOTA 62.0; DanceTrack 69.9 (73.4 с дополнительными данными) — против 47.7 у ByteTrack.
- MOTIP (CVPR 2025): ID как классификация — по истории треков сеть предсказывает номер для каждой новой детекции; DanceTrack HOTA около 71–74.

6. SOT: SiamFC Bertinetto et al., 2016



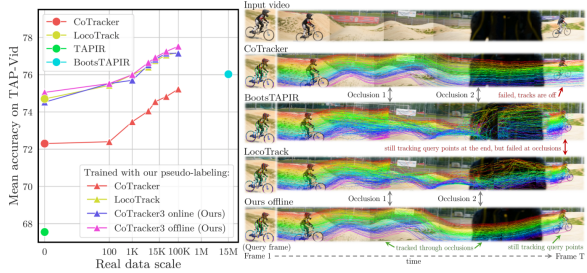
SiamFC: карта откликов

$$f(z,x) = \varphi(z) \star \varphi(x) + b, \qquad \big(\varphi(z) \star \varphi(x)\big)[u,v] = \sum_{c,p,q} \varphi(z)_{c,p,q}\ \varphi(x)_{c,\,u+p,\,v+q}$$

z — шаблон (объект в первом кадре), x — область поиска, φ — общая CNN, b — сдвиг; c, p, q — канал и позиция в шаблоне. Шаблон работает как ядро свёртки; максимум карты — новое положение.

SOT следует за одним заданным объектом. Развитие: SiamRPN (головы бокса), OSTrack (один поток ViT; GOT-10k AO 73.7).

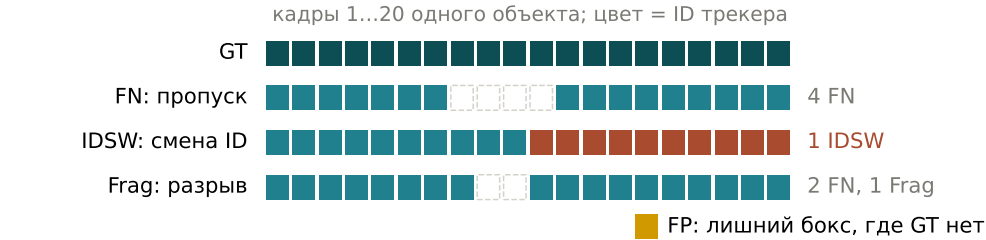
6. Трекинг точек: CoTracker3 Karaev et al., 2024



Karaev et al., CoTracker3, Fig. 1

- Задача: для набора точек в кадре 0 — положение и видимость в каждом следующем кадре, в том числе сквозь окклюзии.
- Трансформер обрабатывает все точки вместе: соседние точки подсказывают, где быть закрытой.
- Отличие от потока (лекция 2): поток плотный, но на один шаг; трекинг точек редкий, но на сотни кадров. Обучение на псевдоразметке реальных видео — в 1000 раз меньше данных.

7. Какие ошибки бывают у трекера



Ошибки по кадрам (боксы сопоставляются с GT при IoU ≥ 0.5)

FN (false negative) — объект есть, бокса нет. **FP** (false positive) — бокс есть, объекта нет.
IDSW (identity switch) — объект сопоставлен с другим ID, чем в прошлый раз. **Frag** — трек прервался и продолжился.

Первые две ошибки — про детекцию, вторые две — про ассоциацию. Метрики различаются тем, как они взвешивают эти группы.

7. MOTA и MOTP: считаем события

MOTA (multi-object tracking accuracy)

$$\mathrm{MOTA} = 1 - \frac{\sum_t \left(\mathrm{FN}_t + \mathrm{FP}_t + \mathrm{IDSW}_t\right)}{\sum_t \mathrm{GT}_t}$$

GT_t — число объектов в кадре t. Доля «нештатных событий» на объект GT; может быть отрицательной, если FP много.

MOTP (precision)

$\mathrm{MOTP} = \dfrac{\sum_{t,i} d_t^i}{\sum_t c_t}$ — средняя мера совпадения d (в MOTChallenge — IoU) по всем c_t сопоставленным парам. Только точность боксов, про ID — ничего.

Недостаток MOTA: IDSW обычно на два порядка меньше FN, так что метрикой правит детектор. И IDSW — это событие, а не длительность: перепутать ID на один кадр и вернуть стоит 2, потерять навсегда — 1.

7. IDF1: сначала сопоставить треки целиком

Глобальное сопоставление идентичностей

Каждому GT-треку ставим в пару *один* предсказанный трек на всё видео — задача о назначениях между треками (раздел 4), где выигрыш пары — число кадров, в которых они совпадают (IoU ≥ 0.5).

IDF1

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}}$$

IDTP — кадры, где GT покрыт *своим* сопоставленным треком; IDFN — кадры GT без своего трека (пропуск или чужой ID); IDFP — кадры предсказанного трека без своего GT. Это F1-мера по кадрам.

Смена ID посреди трека стоит половины трека: вторая половина достаётся другому ID и считается ошибкой. IDF1 — про идентичность, но слабо реагирует на точность боксов.

7. HOTA, часть 1: детекция

Luiten et al., IJCV 2021

HOTA (higher order tracking accuracy) считает детекцию и ассоциацию отдельно и честно объединяет. Всё — при фиксированном пороге локализации α.

DetA: точность детекции при пороге α

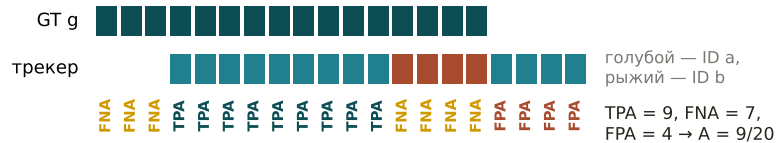
В каждом кадре предсказания сопоставляются с GT один к одному при IoU $\geq \alpha$. Сопоставленная пара — TP, лишнее предсказание — FP, несопоставленный GT — FN.

$$\mathrm{DetA}_\alpha = \frac{|\mathrm{TP}|}{|\mathrm{TP}| + |\mathrm{FN}| + |\mathrm{FP}|}$$

Суммы по всем кадрам видео. Это индекс Жаккара (IoU) множеств «найденное» и «истинное».

DetA ничего не знает об ID: трекер с идеальными боксами и случайными номерами получит $\mathrm{DetA} = 1$. Номера оценивает вторая часть.

7. HOTA, часть 2: ассоциация



Для каждой TP-пары c = (GT g, предсказанный ID a) в каком-то кадре

TPA(c) — кадры, где g сопоставлен именно с a; **FNA**(c) — кадры g с другим ID или без бокса; **FPA**(c) — кадры a на другом объекте или без GT.

$$\mathcal{A}(c) = \frac{|\mathrm{TPA}(c)|}{|\mathrm{TPA}(c)| + |\mathrm{FNA}(c)| + |\mathrm{FPA}(c)|}, \qquad \mathrm{AssA}_\alpha = \frac{1}{|\mathrm{TP}|}\sum_{c \in \mathrm{TP}} \mathcal{A}(c)$$

$\mathcal{A}(c)$ — насколько траектории g и a совпадают целиком (снова индекс Жаккара, но по кадрам двух треков). На рисунке $\mathcal{A} = 9/20$.

7. HOTA, часть 3: собираем метрику

HOTA

$$\text{HOTA}_\alpha = \sqrt{\text{DetA}_\alpha \cdot \text{AssA}_\alpha}, \qquad \text{HOTA} = \frac{1}{19} \sum_{\alpha \in \{0.05, 0.10, \ldots, 0.95\}} \text{HOTA}_\alpha$$

- **Геометрическое среднее**: если одна часть близка к нулю, близка к нулю и метрика. Хорошая детекция не «покупает» плохую ассоциацию.
- **Среднее по 19 порогам** α встраивает точность боксов: при $\alpha = 0.95$ TP остаются только идеальные боксы. Отдельный MOTP не нужен.
- Компоненты публикуют отдельно (DetA, AssA, LocA) — видно, где у трекера проблема.

Ограничения: HOTA не различает онлайн и офлайн трекеры и не штрафует задержку ответа. Для продакшена смотрите ещё FPS и latency.

7. Один GT-трек, два трекера: метрики расходятся

Трекер A: видит все 20 кадров, 1 смена ID



MOTA = 1 − (0 + 0 + 1)/20 = 0.95

IDF1 = 2·10 / (2·10 + 10 + 10) = 0.50

DetA = 20/20 = 1.00

AssA = 10/(10 + 10 + 0) = 0.50

HOTA = √(1.00 · 0.50) = 0.71

Трекер B: видит 12 кадров из 20, ID один



MOTA = 1 − (8 + 0 + 0)/20 = 0.60

IDF1 = 2·12 / (2·12 + 0 + 8) = 0.75

DetA = 12/20 = 0.60

AssA = 12/(12 + 8 + 0) = 0.60

HOTA = √(0.60 · 0.60) = 0.60

MOTA почти не замечает смену ID у A (0.95) и сильно ругает пропуски B (0.60). IDF1 предпочитает B ($0.75 > 0.50$). HOTA учитывает обе ошибки: у B пропуски портят и DetA, и AssA — незамеченный объект и не ассоциирован.

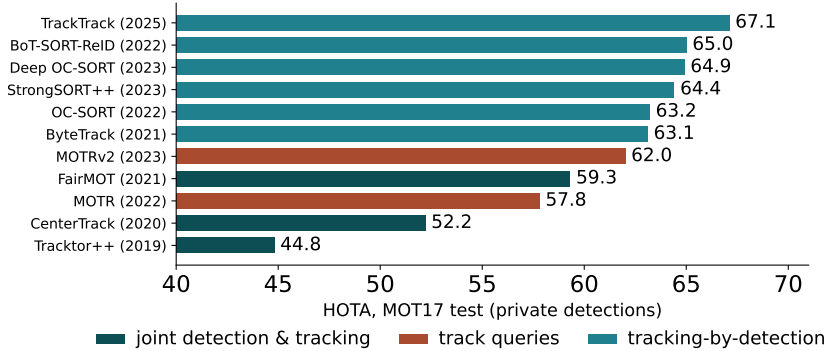
7. Датасеты MOT

Датасет	Видео	Кадры	Треки	Боксы	Чем сложен
MOT17 (2017)	14 (7/7)	11 235	1 342	293 тыс.	пешеходы; часть камер движется
MOT20 (2020)	8 (4/4)	13 410	3 833	2.1 млн	толпа: ≈157 боксов на кадр
DanceTrack (2022)	100 (40/25/35)	105 855	990	877 тыс.	одинаковые костюмы, резкое движение
SportsMOT (2023)	240	>150 тыс.	—	1.6 млн	баскетбол, волейбол, футбол

- В скобках — разбиение train/test (у DanceTrack ещё val).
- MOT17 почти насыщен: лучшие трекеры различаются на 1–2 HOTA. Прогресс ассоциации виден на DanceTrack и SportsMOT, где детекция простая.

motchallenge.net; dancetrack.github.io; Cui et al., SportsMOT, ICCV 2023

7. MOT17: прогресс за шесть лет



MOT17 test, private detections; таблицы статей MOTRv2 и TrackTrack

От Tracktor++ (44.8) до TrackTrack (67.1) — $+22$ HOTA. Лидирует tracking-by-detection: на пешеходах MOT17 сильный детектор плюс аккуратная ассоциация пока выигрывают у end-to-end.

8. Где трекеры ломаются

Ситуация	Что происходит	Чем лечат
окклюзии, пересечения	смена ID, разрывы	буфер lost, ReID, OC-SORT
движение камеры	прогноз Калмана уезжает	CMC (BoT-SORT), поток
толпа (MOT20)	близкие стоимости, конфликты	венгерский, каскад, гейт
одинаковая внешность	ReID бесполезен	движение, MOTRv2, MOTIP
нелинейное движение	постоянная скорость неверна	адаптивные Q, R, обучаемые модели движения
низкий FPS, дроны	большие сдвиги, IoU $= 0$	Махаланобис, расширенный бокс в IoU

Каждая строка — одна идея из раздела 5.

8. Состояние области в 2026

- **Онлайн TbD на MOT17**: TrackTrack (HOTA 67.1); FastTracker (2025) — 66.4 на MOT17 и 65.7 на MOT20.
- **Инструменты**: Ultralytics 8.4.63+ — шесть трекеров в одном вызове `model.track()` (BoT-SORT, ByteTrack, OC-SORT, Deep OC-SORT, FastTracker, TrackTrack), по умолчанию TrackTrack. YOLO26 (январь 2026) — детектор без NMS.
- **Честное сравнение**: библиотека Roboflow `trackers` (один детектор для всех); TrackEval — эталонная реализация HOTA.
- **End-to-end**: на DanceTrack лучшие результаты у MOTRv2, MOTIP и McByte (67.2 HOTA при детекциях YOLOX).
- **Tracking-by-segmentation**: SAM2MOT — маски и память SAM 2 вместо Калмана (лекция 6).
- **Открытый вопрос**: универсальная модель движения — трекинг точек и обучаемые фильтры против постоянной скорости.

8. Как выбрать трекер на практике

Быстрый старт

Сильный детектор + ByteTrack. Без ReID, сотни кадров в секунду на CPU для самого трекера.

Двигается камера

BoT-SORT (компенсация движения) или OC-SORT.

Долгие окклюзии, разная внешность

Добавить ReID: DeepSORT, BoT-SORT-ReID, Deep OC-SORT.

Одинаковые объекты

Спорт, танцы, резкое движение: OC-SORT как база; если есть данные и GPU — MOTRv2 или MOTIP.

Как сравнивать

Один детектор для всех; HOTA вместе с DetA и AssA; IDF1 и число IDSW; FPS на целевом железе.

Главные выводы

1. Трекинг добавляет к детекции одну вещь — номер объекта, постоянный во времени
2. Время можно использовать в детекторе (Seq-NMS, FGFA), но сегодня его обычно отдают трекеру
3. Фильтр Калмана — это оптимальное сложение прогноза и детекции: $k = \sigma_a^2/(\sigma_a^2 + \sigma_z^2)$ в 1D и $K = P^- H^\top S^{-1}$ для векторов
4. Ассоциация — задача о назначениях: стоимости из IoU, Махаланобиса и внешности, точное решение венгерским алгоритмом, затем порог
5. SORT $\to$ DeepSORT $\to$ ByteTrack $\to$ OC-SORT, BoT-SORT $\to$ TrackTrack: один цикл, каждая идея лечит один провал
6. End-to-end трекеры (MOTRv2, MOTIP) выигрывают там, где ассоциация сложнее детекции
7. HOTA $= \sqrt{\text{DetA} \cdot \text{AssA}}$: у MOTA ошибки детекции перевешивают, у IDF1 — ошибки идентичности; сравнивать трекеры — на одном детекторе

Литература (1/2)

- **Bewley et al.** — Simple Online and Realtime Tracking (SORT), ICIP 2016. arxiv.org/abs/1602.00763
- **Wojke, Bewley, Paulus** — DeepSORT, ICIP 2017. arxiv.org/abs/1703.07402
- **Zhang et al.** — ByteTrack, ECCV 2022. arxiv.org/abs/2110.06864
- **Cao et al.** — Observation-Centric SORT, CVPR 2023. arxiv.org/abs/2203.14360
- **Aharon, Orfaig, Bobrovsky** — BoT-SORT, 2022. arxiv.org/abs/2206.14651
- **Shim et al.** — Focusing on Tracks for Online MOT (TrackTrack), CVPR 2025

Литература (2/2)

- **Zhu et al.** — Flow-Guided Feature Aggregation, ICCV 2017. arxiv.org/abs/1703.10025; **Han et al.** — Seq-NMS, 2016
- **Zhou, Koltun, Krähenbühl** — CenterTrack, ECCV 2020; **Zeng et al.** — MOTR, ECCV 2022
- **Zhang et al.** — MOTRv2, CVPR 2023. arxiv.org/abs/2211.09791; **Gao et al.** — MOTIP. arxiv.org/abs/2403.16848
- **Bertinetto et al.** — SiamFC, ECCVW 2016; **Karaev et al.** — CoTracker3, 2024. arxiv.org/abs/2410.11831
- **Luiten et al.** — HOTA, IJCV 2021; TrackEval
- Roboflow trackers: trackers.roboflow.com; Ultralytics Track: docs.ultralytics.com/modes/track